

# Complete Guide: Stack, Heap and Dangling Pointers in C and C++

Study notes covering memory layout, lifetime, allocation, memory leaks, RAI and modern C++ best practices.

## 1. Memory Layout

Typical simplified view:

```
High Addresses
+-----+
|      Stack      |
+-----+
|   Free Space   |
+-----+
|      Heap      |
+-----+
Low Addresses
```

## 2. Stack Memory

The stack stores function call frames and usually local variables. Allocation and deallocation are automatic. When a function returns, its stack frame is destroyed.

## 3. Heap Memory

The heap stores dynamically allocated objects. The programmer controls the lifetime directly in C and traditionally through new/delete in C++.

### C Stack Example

```
void f()
{
    int x = 42;
    double y = 5.5;
}
```

### C Heap Example

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);
```

### C++ Heap Example

```
int* p = new int(42);
delete p;
```

### Bad: Returning Address of Local Variable

```
int* bad()
{
    int x = 42;
    return &x;
}
```

### Good: Object Survives Function Return

```
int* good()
{
    int* p = new int(42);
    return p;
}
```

```
int* result = good();
delete result;
```

### Dangling Pointer After delete

```
int* p = new int(42);
delete p;
std::cout << *p; // Undefined behavior
```

## Memory Leak

```
int* p = new int(42);
// delete forgotten
```

## Modern C++ RAI

```
auto p = std::make_unique<int>(42);
// released automatically
```

## std::vector

```
std::vector<int> v(1000);
// vector manages heap memory internally
```

## shared\_ptr

```
auto p1 = std::make_shared<int>(42);
auto p2 = p1;
```

## Why the Heap Exists

A major reason for heap allocation is lifetime management. Local stack variables disappear when a function returns. Heap-allocated objects can continue existing after the creating function has ended, allowing their address to be returned safely.

## Important Nuance About Dangling Pointers

Heap allocation prevents the specific error of returning the address of a local stack variable. However, heap memory does not eliminate dangling pointers. Any pointer becomes dangling once the object it points to has been destroyed or freed.

## Stack vs Heap Comparison

| Feature       | Stack        | Heap                                   |
|---------------|--------------|----------------------------------------|
| Allocation    | Automatic    | Manual or managed                      |
| Lifetime      | Scope based  | Until free/delete or owner destruction |
| Speed         | Very fast    | Typically slower                       |
| Size          | Limited      | Larger                                 |
| Typical Risks | Scope errors | Leaks and dangling pointers            |

## Key Takeaways

- Stack variables are automatically destroyed when their scope ends.
- Heap objects survive function returns and are useful for dynamic lifetime management.
- Returning addresses of local variables creates dangling pointers.
- Using a pointer after free() or delete creates a dangling pointer.
- Forgetting free() or delete causes memory leaks.
- Modern C++ prefers RAI, std::vector, std::string, std::unique\_ptr and std::shared\_ptr.
- A pointer is valid only while the object it points to is alive.